



FILE PATH TRASVERSAL, TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

ROAD TO HALL OF FAME



BRYHAN EDUARDO GUZMAN MARTINEZ

178508

SEGURIDAD INFORMÁTICA

ÍNDICE

Marco Teórico y Fundamentación Técnica.....	2
Explicación Técnica de la Vulnerabilidad.....	2
Explicación Técnica del Payload.	2
Evidencias y Capturas de Pantalla	3
Mitigación Recomendada	7

Marco Teórico y Fundamentación Técnica

- **Vulnerabilidad:** El *File Path Traversal* (Salto de Directorio) es una vulnerabilidad web que permite a un atacante leer archivos arbitrarios en el servidor que ejecuta una aplicación.
- **Relación con OWASP:** Esta vulnerabilidad se clasifica típicamente dentro de la categoría **Broken Access Control** del OWASP Top 10, ya que permite evadir las restricciones de acceso al sistema de archivos.
- **Impacto en la Triada CIA:** Afecta directamente la **Confidencialidad** (C), ya que expone datos sensibles, credenciales o archivos del sistema (como `/etc/passwd`) a usuarios no autorizados.

Explicación Técnica de la Vulnerabilidad

La aplicación implementa un mecanismo de defensa deficiente para evitar ataques de Path Traversal. Intenta sanitizar el input del usuario eliminando las secuencias de salto de directorio (`../`). Sin embargo, esta sanitización se realiza de manera **no recursiva**. Esto significa que el filtro pasa una sola vez sobre la cadena y elimina el patrón encontrado, pero no vuelve a verificar si la cadena resultante ha formado un nuevo patrón malicioso.

Explicación Técnica del Payload.

- **Funcionamiento del Payload:** Se utilizó el payload `....//....//....//etc/passwd` porque el servidor elimina la secuencia `../`.
- Al enviar `....//`, el filtro identifica y elimina los caracteres `../` del medio.
- Los caracteres restantes (el primer par de puntos y la última barra) se concatenan automáticamente, formando nuevamente la secuencia `../`.
- Al repetir esto tres veces, logramos retroceder tres niveles en la estructura de directorios hasta llegar a la raíz (root), permitiendo acceder a `/etc/passwd`.

Evidencias y Capturas de Pantalla

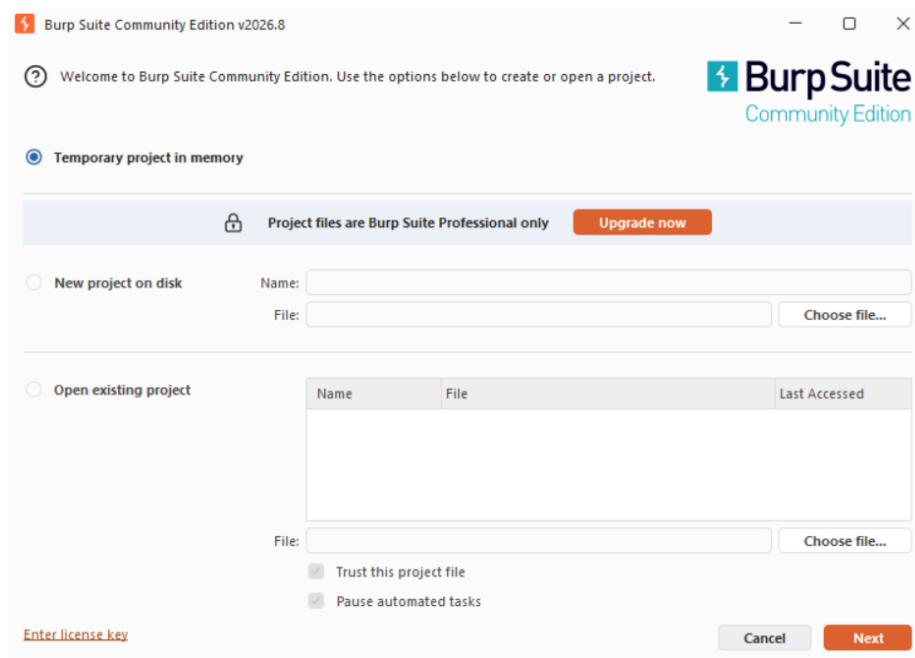


Imagen 1. Abrimos BurpSuit previamente descargado, seleccionamos “Temporary Project in memory”, después demos click en “Next”

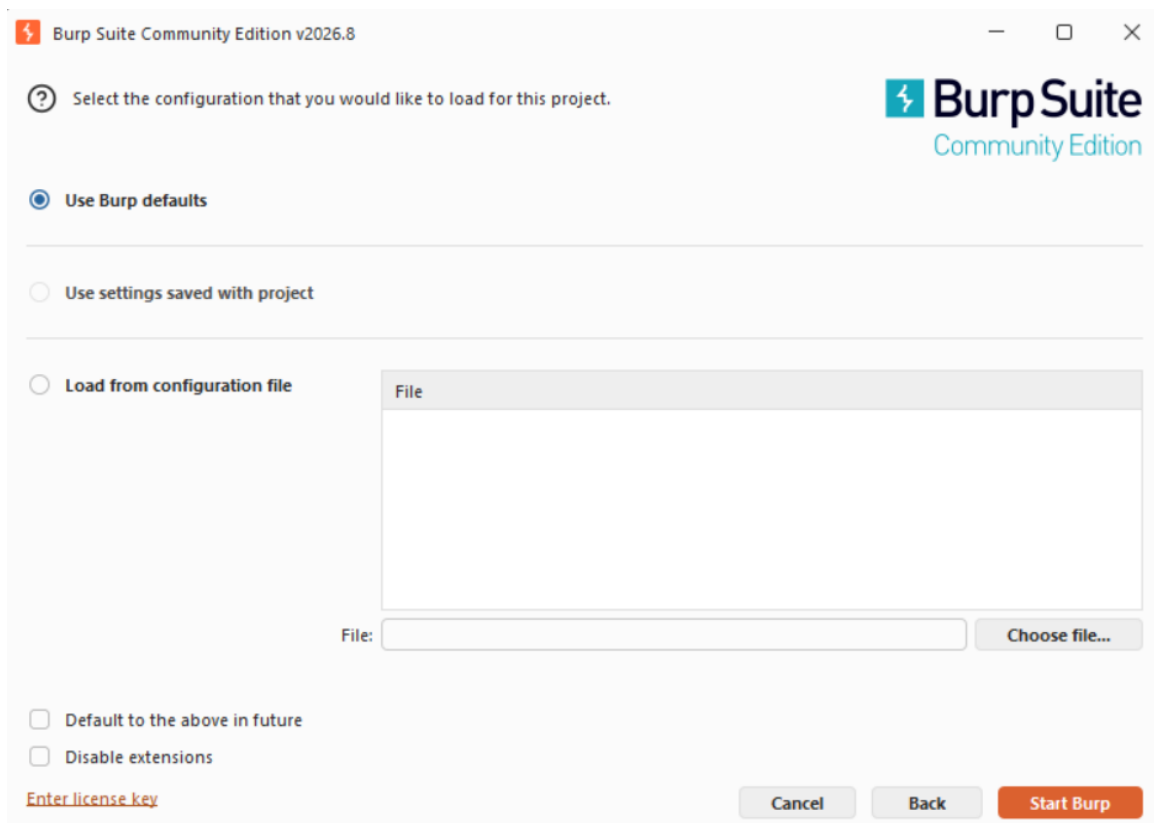


Imagen 2. Seleccionamos “Use Burp defaults” y damos click en “Start Burp”.

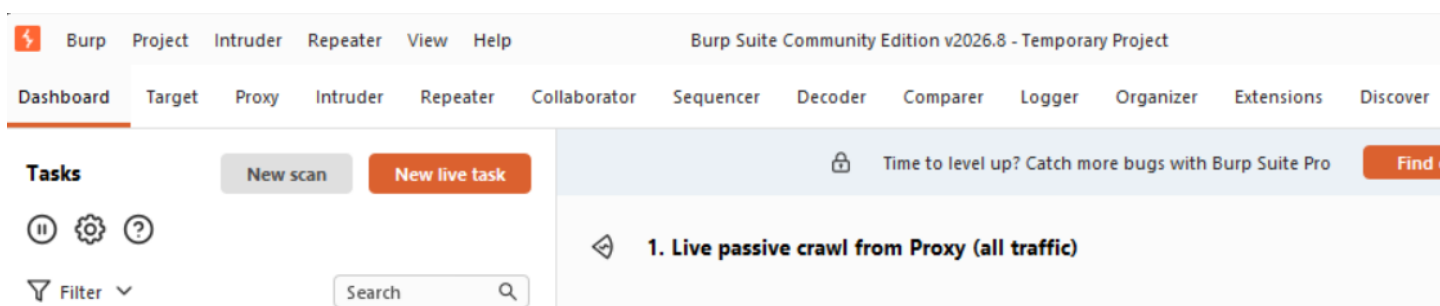


Imagen 3. En las pestañas que están arriba, daremos click en “Proxy”

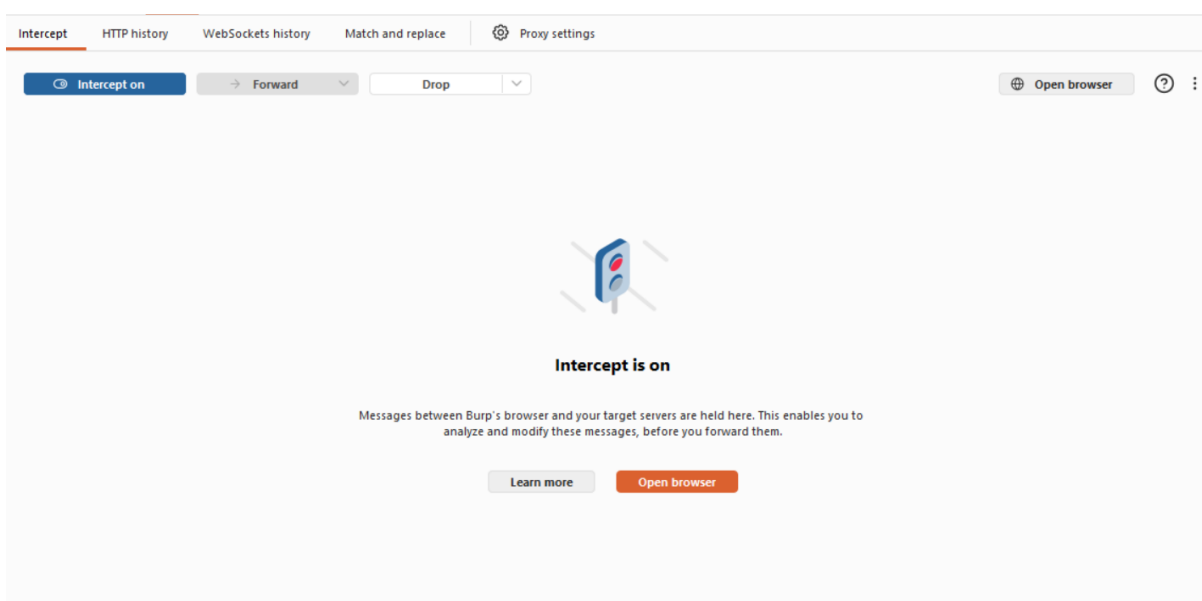


Imagen 4. Daremos click en “Intercept off” para que cambie a “Intercept on” después daremos click en open browser.

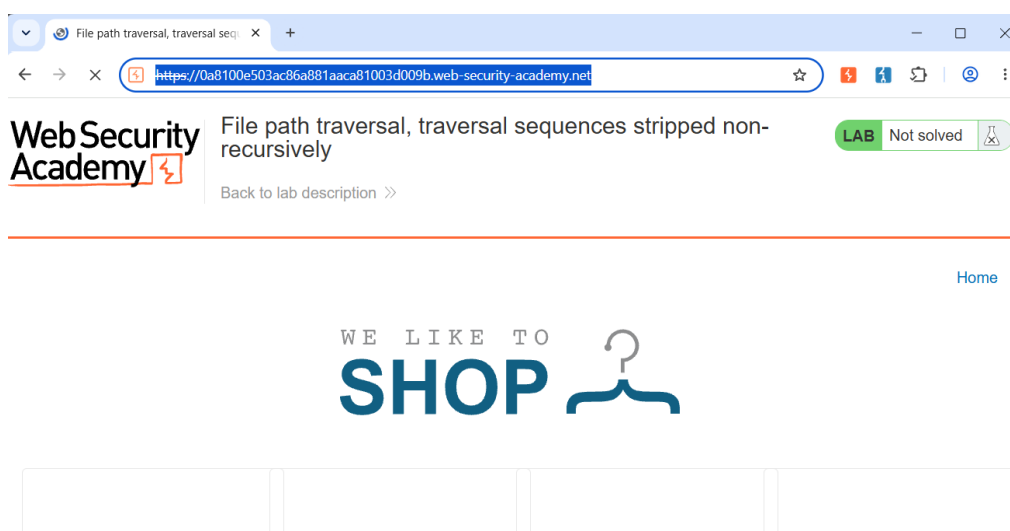


Imagen 5. Se abrirá una pestaña del navegador donde pegaremos la URL que nos dieron cuando accedimos al laboratorio

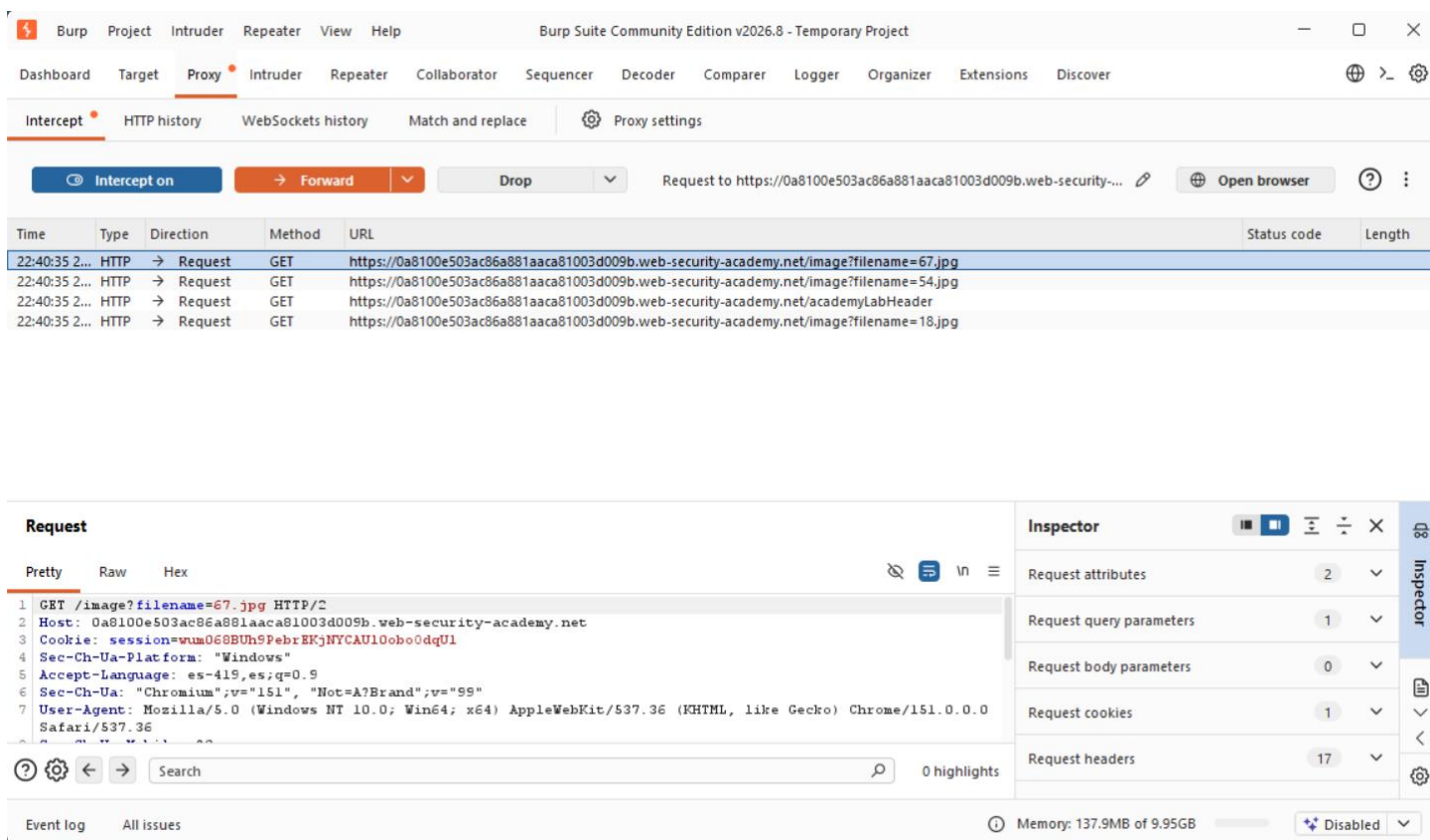


Imagen 6. En BurpSuite nos aparecerá un archivo HTTP, donde presionaremos “FOWARD” después se abrirán más archivos, el que ocuparemos será el que dice “GET /image?filename=67.jpg”.

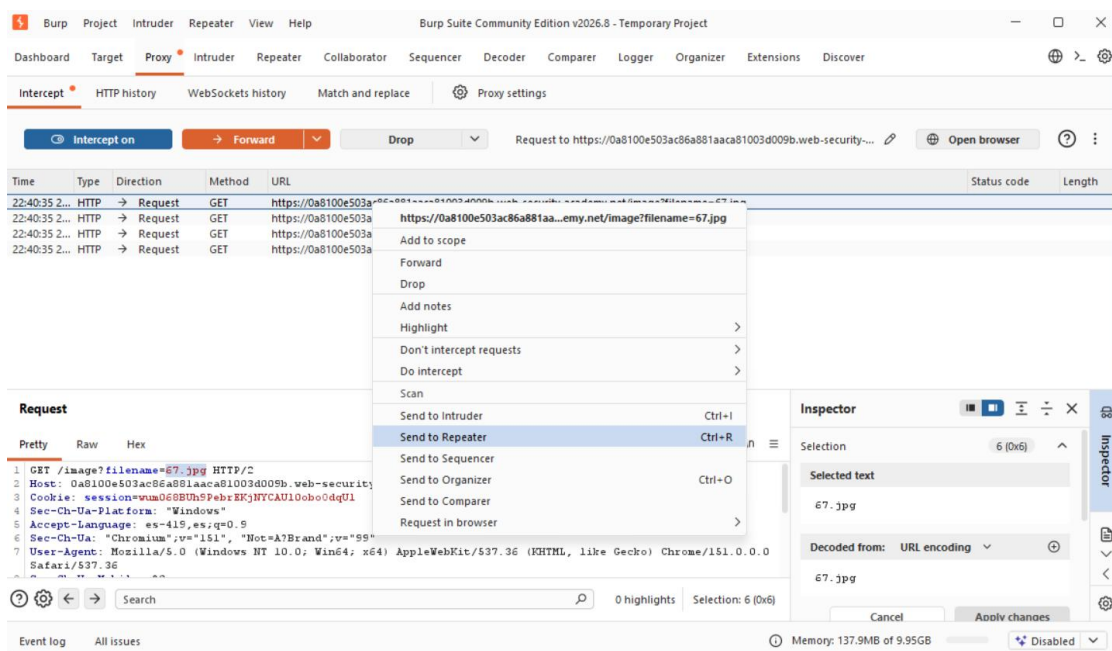


Imagen 7. El cual seleccionaremos daremos click y pondremos la opción “Send to Repeater”

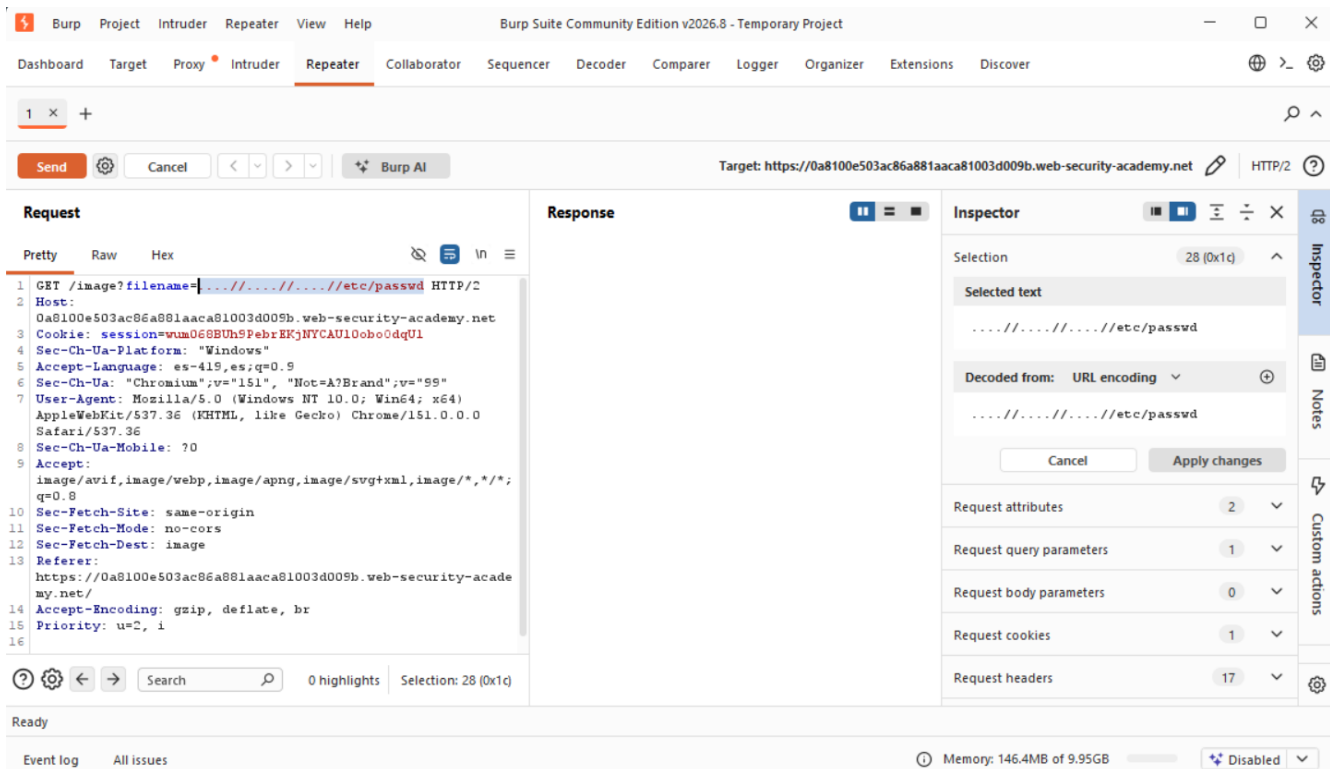


Imagen 8. Cambiaremos el nombre de la ruta de “GET /image?filename=67.jpg” por “GET /image?filename=.....//etc/passwd” y daremos click en “Send”.

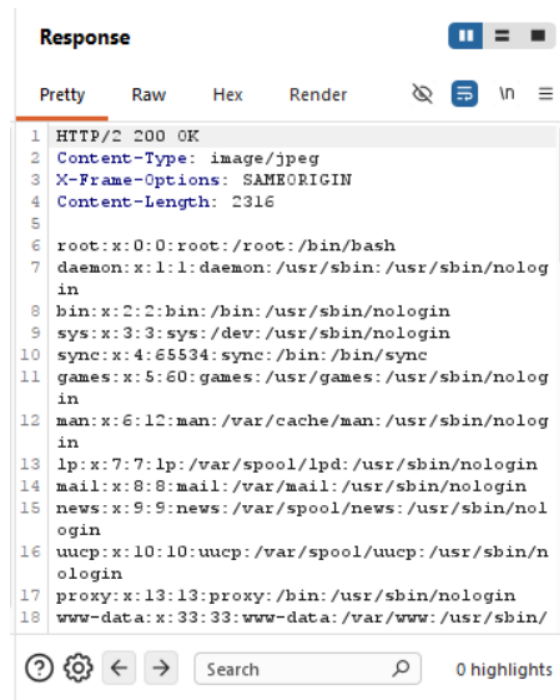


Imagen 9. Una vez que se envió aparecerá que todo está bien con un “200 OK” y con toda la información por la cual hicimos el ataque.

Lab: File path traversal, traversal sequences stripped non-recursively

PRACTITIONER



LAB

✓ Solved



This lab contains a path traversal vulnerability in the display of product images.

The application strips path traversal sequences from the user-supplied filename before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

💡 Solution



💡 Community solutions



Imagen 10. En la página del laboratorio aparecerá que ya está resuelto ese laboratorio (Solved ✓)

Mitigación Recomendada

Para mitigar esta vulnerabilidad de manera efectiva, se recomiendan las siguientes prácticas de defensa:

- **Evitar el input del usuario en APIs del sistema de archivos:** En la medida de lo posible, no pasar directamente nombres de archivos proporcionados por el usuario a funciones del sistema.
- **Uso de Listas Blancas (Allowlisting):** Si es estrictamente necesario, validar el input contra una lista estricta de archivos permitidos o usar identificadores alfanuméricos (como un ID de base de datos) en lugar del nombre del archivo.
- **Validación de ruta canónica:** Antes de acceder al archivo, la aplicación debe resolver la ruta absoluta (usando funciones como `getCanonicalPath()` en Java, por ejemplo) y verificar que esta ruta validada comience exactamente con el directorio base esperado.